Predicting Track Popularity on Last.fm with Linear Regression

Dana Belgibek
SDU University, SDS307 Modern Regression Analysis

October 2026

Abstract

We study which factors are linked to the number of plays (`playcount`) of music tracks on Last.fm. We collected 12,035 tracks by 2,514 artists with the public Last.fm API and made 23 features about the title, the track and the artist. A linear regression for $\log(1+\texttt{playcount})$ reaches a test R^2 of 0.71 on artists the model has not seen. The artist's audience and the track information explain most of it; the title explains almost nothing. Diagnostics show unequal error variance and repeated features, so we removed two features.

1 Introduction and Overview

Motivation. The idea came from a simple question: when we discover a new artist, which of their songs should we listen to first, without listening to all of them? Personal taste is hard to measure, so we changed the question to popularity: which factors are associated with the number of plays of a track. Such a model could help new artists and producers to see which factors are linked to popular tracks, for example a large artist audience or a track with tags and a description on Last.fm. A new artist cannot quickly grow the audience, but can fill in the track information. The model shows associations, not causes, so it can support such decisions but cannot guarantee popularity.

Problem and data. We predict the number of plays of a track from 23 features. The data come from the public Last.fm API (collected on October 7, 2026): 12,035 tracks by 2,514 artists from 14 genre tags.

Target. `playcount` is the number of plays of a track. It is filled in for all tracks. The median is 1,426,212 and the maximum is 123,075,880, so a few tracks are much bigger than the rest (skewness 4.22). After the log transformation, $\log(1+\texttt{playcount})$ has skewness -0.33 and goes from 3.7 to 18.6 (Figure 1), so the models use the log target.

2 Theoretical Background

We use multiple linear regression, $y_i = \beta_0+\beta_1 x_{i1}+\cdots+\beta_p x_{ip}+\varepsilon_i$. Ordinary least squares (OLS) chooses the coefficients that make the sum of squared errors $\sum_i (y_i-\hat{y}_i)^2$ as small as possible. The model assumes a linear relation, independent errors, the same error variance everywhere and normal errors. To compare models we use

$$R^2 = 1-\frac{\sum_i (y_i-\hat{y}_i)^2}{\sum_i (y_i-\bar{y})^2}, \quad R^2_{adj} = 1-(1-R^2)\frac{n-1}{n-p-1}, \quad \mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_i (y_i-\hat{y}_i)^2}, \quad \mathrm{MAE} = \frac{1}{n}\sum_i |y_i-\hat{y}_i|. \tag{1}$$

R^2 is the share of the variation that the model explains; adjusted R^2 also punishes extra features. RMSE and MAE show the typical size of an error (RMSE punishes large errors more). Features that are strongly related to each other (multicollinearity) are found with $\mathrm{VIF}_j = 1/(1-R^2_j)$, where R^2_j comes from predicting feature j from the other features; a large VIF means that the feature repeats the others. We check equal variance with a residual plot and the Breusch–Pagan test, and normality with a Q-Q plot. A categorical feature with k categories becomes $k-1$ dummy (0/1) variables; the missing one is the reference category, which avoids the dummy variable trap.

3 Algorithm Implementation and Development

3.1 Data collection and ethics

First we tried the Spotify Web API, but its audio-features endpoint returned error 403 for our app, so we moved to Last.fm. We used three official API methods: `tag.getTopTracks` to get the top tracks of each genre tag (up to 20 pages of 50 tracks), `track.getInfo` to get the plays, duration, album, tags and description of each track, and `artist.getInfo` to get the listeners, plays and biography of each artist. We used 14 tags (indie, alternative, electronic, hip-hop, rock, jazz, metal, folk, punk, soul, r&b, reggae, country and blues) and stopped at 12,037 candidate tracks. After every request the script waits 1 second, failed requests are repeated, and the results are saved after each request, so the collection can continue after a break (`scraper.py`). We kept the raw data before cleaning unchanged.

We used only documented API methods with a registered key and a clear User-Agent, and we did not scrape any website pages. On October 11, 2026 we checked `robots.txt` of `www.last.fm`: it blocks only website pages such as `/search`, `/forum` and `/label`; for the API host the file could not be opened. We also read the API Terms of Service [5]. The data (about 2 MB, below the 100 MB limit) are used only for this course. The Terms ask academic users to contact Last.fm first, which we did not do. We did not collect any data about Last.fm users, and in the shared file the names are replaced by numbers.

3.2 Cleaning and features

After merging and removing records without an API answer, 12,035 of the 12,037 tracks remained; there were no duplicates and no zero play counts. Of the 32 raw features we removed nine (`cleaning.py`): four title flags that are almost always 0 (`has_remix`, `has_live`, `has_edit`, `has_version`), four features with (almost) the same value for all tracks (`n_similar_artists`, `n_artist_tags`, `album_position`, `has_artist_bio`) and `top_tag`, which is "none" for 80.8% of the tracks. It seems that the API always returns the same number of similar artists and tags, so these features were useless; this part of the collection did not work. `duration_min` has 321 missing values (2.7%); we fill them with the median of the training set after the split. Table 1 lists the 23 features that remain.

Table 1: The 23 features after cleaning (identifiers and target not counted).

Group	No.	Content
Track title	7	length, word count, flags for featuring, parentheses, digits, non-ASCII, title equal to album
Artist name	3	length, word count, collaboration flag
Track metadata	5	duration, has album, `n_track_tags`, `has_wiki`, `wiki_length`
Artist statistics	6	listeners, plays, `log_artist_listeners`, plays per listener, biography length, on tour
Sampling	2	`genre_tag` (categorical), `rank_in_genre`

Problems with the data. (i) `rank_in_genre` only goes from 1 to 50 and starts again on every page, so it is the position on a page, not the real rank in the genre; (ii) `n_track_tags` is 0 for 9,729 tracks and 5 for 2,305 (one track has 3), so it only shows whether tags exist; (iii) the tracks come from top lists, so popular tracks are over-represented and the genre sizes depend on how we collected the data (blues has only 255 tracks); (iv) each artist has about 4.8 tracks, so the rows are not fully independent; (v) `artist_playcount` already includes the plays of the track itself (possible leakage).

3.3 Split and model fitting

We split the data 80/20 by artist (`GroupShuffleSplit`, seed 42), so no artist is in both parts: 9,804 tracks of 2,011 artists for training and 2,231 tracks of 503 artists for testing. The target is $\log(1 + \texttt{playcount})$. `artist_playcount` and `wiki_length` are also very skewed, so we use their logarithms; we drop the raw `artist_listeners` and keep its logarithm. `genre_tag` becomes dummy variables with *alternative* as the reference category, which gives 34 predictors. We fit OLS with `statsmodels` and `scikit-learn`. The final model removes two repeated features (Sec. 4.3). The code is in `analysis.ipynb`.

4 Computational Results

4.1 Understanding the data

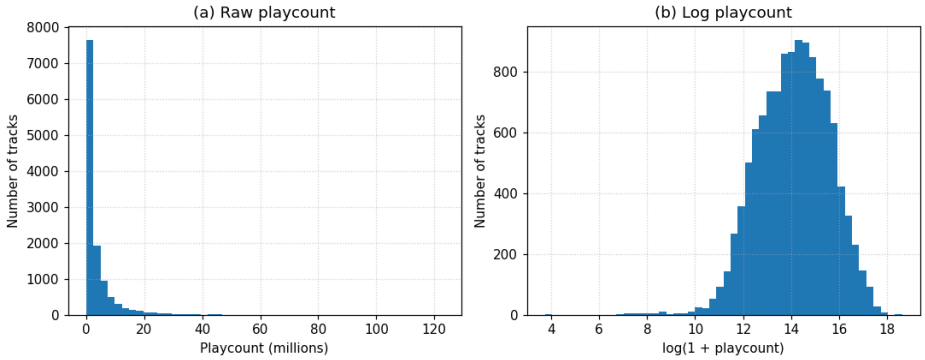


Figure 1: Distribution of `playcount` before (a) and after (b) the log transformation.

The play counts and the artist audiences cover several orders of magnitude (artist listeners from 805 to 9.3 million), so we use logarithms. Most artists have few tracks: 47% have one track and 70% at most three, while the top 10% of artists give 54% of all tracks. The features most correlated with the log target are `log_artist_listeners` (0.64), `n_track_tags` (0.62), `wiki_length` (0.57), `artist_listeners` (0.56) and `has_wiki` (0.55). Title and artist-name features have weak correlations (for example `artist_length` −0.12), and `rank_in_genre` has almost none (−0.025). Only 19% of the tracks have tags, but their median play count is 8.7 million against 0.96 million for the others. Figure 2 shows that artists with a bigger audience have more popular tracks in both groups (correlation 0.60 without tags, 0.36 with tags). The median log play count is highest for rock, alternative and indie (15.4–15.6) and lowest for reggae, blues and jazz (12.5–13.0), but the genre sizes depend on how we collected the data.

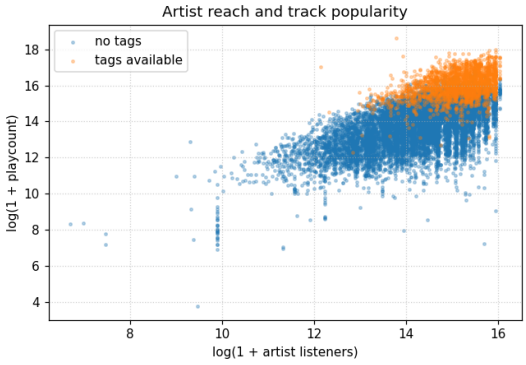


Figure 2: Artist reach and track popularity, by availability of track tags.

4.2 Models

Table 2: Test performance on the log target (2,231 tracks by 503 unseen artists).

Model	Predictors	R^2 test	RMSE test
Baseline (training mean)	0	−0.006	1.473
A: title and artist name	10	0.012	1.460
B: A + artist statistics	15	0.421	1.118
C: B + track metadata	20	0.651	0.868
D: C + genre and rank (full model)	34	0.714	0.786
Final model (D without `has_wiki`, `log_artist_playcount`)	32	0.711	0.790

Table 2 shows that title and artist-name features alone explain almost nothing ($R^2 = 0.012$). Artist statistics add the most, and track information and genre add more. The full OLS model has a training

R^2 of 0.742 and a test R^2 of 0.714 (adjusted 0.709), RMSE 0.786 and MAE 0.611. Since the target is a logarithm, an RMSE of 0.79 means that a prediction is usually off by a factor of about $e^{0.79} \approx 2.2$. The training and test R^2 are close, so there is no strong overfitting. With a random split instead of a split by artist, the test R^2 would be 0.751, which is too optimistic. The possible leakage through the artist's play count is small: without `log_artist_playcount` the test R^2 is 0.711, without also `artist_plays_per_listener` 0.707, and without all artist statistics 0.642.

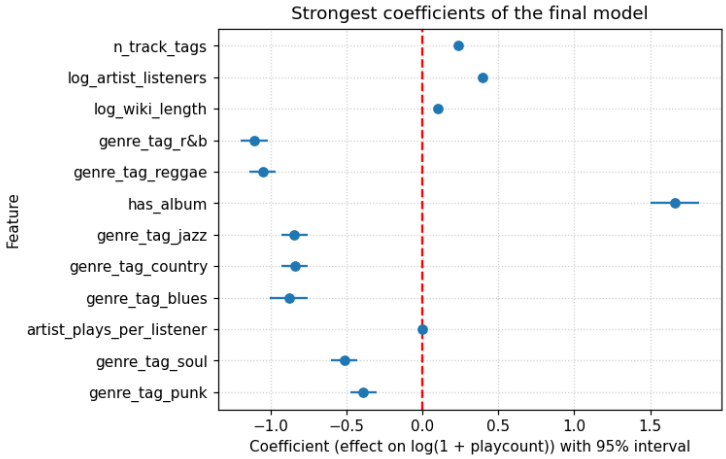


Figure 3: Twelve strongest coefficients of the final model with 95% confidence intervals.

Figure 3 shows the strongest coefficients of the final model. If the other features stay the same, a track with tags (`n_track_tags`, 0.24 per tag, five tags) has about $e^{1.19} \approx 3.3$ times more plays, and an artist audience 10% larger goes with about 4% more plays (coefficient 0.40 of `log_artist_listeners`). These are associations, not causes: tags exist mostly for tracks that are already popular. Compared with the reference genre alternative, play counts are lower for r&b (−1.11), reggae (−1.05), blues (−0.88), jazz and country (both −0.84). Tracks on an album have much higher play counts (coefficient 1.67), but only 126 tracks (1%) have no album, so this result is based on few tracks.

4.3 Diagnostics and unsuccessful results

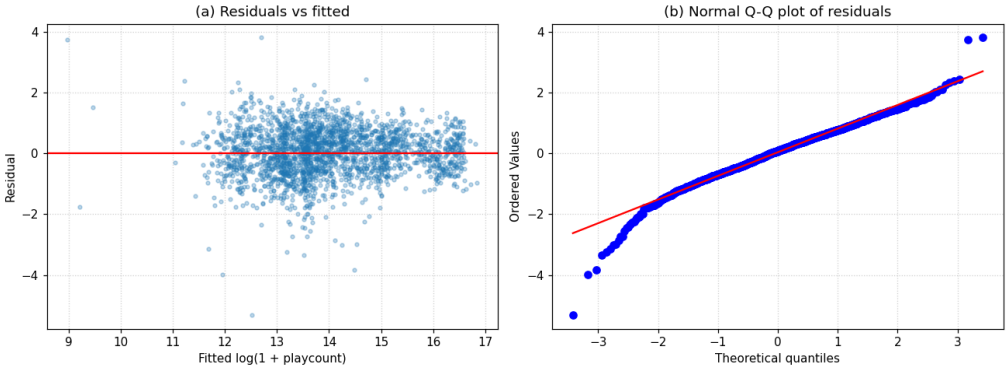


Figure 4: OLS residuals on the test set: residuals vs. fitted values (a) and normal Q-Q plot (b).

- **Unequal error variance (heteroscedasticity):** the Breusch–Pagan test says that the error variance is not the same everywhere (LM = 1919, $p < 0.001$), so the standard errors and p-values should be read with care.
- **Non-normal residuals:** the residuals are skewed to the left (skewness −0.47) and have heavy tails (excess kurtosis 2.28), Figure 4(b).
- **Multicollinearity:** VIF is 70.6 for `log_wiki_length`, 68.7 for `has_wiki`, 23.1 for `log_artist_playcount` and 16.2 for `log_artist_listeners`. Because of this, `has_wiki` gets a negative coefficient (−0.85) although its correlation with the target is positive (0.55). Without `has_wiki` and

`log_artist_playcount` the largest VIF is 6.4 and the test R^2 is 0.711, so these two features only repeated others.

- **Raw skewed predictors:** our first model used the raw `artist_listeners` and `artist_playcount`; both coefficients were negative although the correlations are positive. Using logarithms fixed this.
- **Outliers:** a few tracks have very low play counts (single points in Figures 2 and 4a), and six training tracks are longer than 20 minutes; we kept them in the model.

5 Summary and Conclusions

Best model. The final linear model for the log play count (32 predictors) reaches a test R^2 of 0.71 and RMSE 0.79 on artists it has not seen.We prefer it to the full model D (test R^2 0.714) because its coefficients are reliable (largest VIF 6.4 instead of 70.6) at almost the same accuracy. The artist matters most: tracks of artists with a large audience have many more plays, and artist statistics raise the test R^2 from 0.01 to 0.42 (Table 2). Track information on Last.fm (tags, description, album) adds the next largest part (from 0.42 to 0.65). The title of a track has almost no effect. Genres differ in play counts, but this may partly come from the way the sample was collected.

Back to the starting question. Our first idea was to find which songs of a new artist to listen to first. The data contain no information about personal taste, so the model can only describe popularity, not what a single listener will like.

Recommendations. For a new artist: (1) the size of the audience is the strongest factor, much stronger than the title; (2) complete track information (tags, description, album) goes together with popular tracks, although popular tracks may simply receive more tags; (3) with a typical error of a factor of 2.2, the model is suitable for a rough comparison of tracks, not for exact forecasts. Features that contain the target, such as the artist's play count, should be avoided.

How the course methods helped. The log transformation made the skewed target suitable for linear regression, and dummy variables allowed us to use the genre. The split by artist, adjusted R^2, RMSE and MAE gave an honest estimate of accuracy. VIF, residual and Q-Q plots and the Breusch–Pagan test showed where the assumptions fail and led us to remove two redundant features.

Limits and extensions. The tracks come from top lists of tags, so the results do not describe all tracks, and the effects are associations, not causes. The error variance is not constant and the residuals are not normal. The data are one snapshot (October 7, 2026) without release dates, while musical tastes change quickly, so the model cannot tell whether a new song will succeed. Possible extensions: data collected at several dates with release dates, richer track data from the API (for example full tag lists) instead of the constant features, and a random sample of tracks instead of top lists.

References

[1] Last.fm API documentation, `https://www.last.fm/api`. Accessed October 2026.

[2] Dataset (CSV, names replaced by identifiers): `https://drive.google.com/file/d/136-UX_rI5N2WoEPHlo5aL3l2i0R0QwI5/view?usp=sharing`.

[3] Pedregosa et al., Scikit-learn: Machine Learning in Python, JMLR 12, 2011.

[4] Seabold and Perktold, Statsmodels: Econometric and statistical modeling with Python, 2010.

[5] Last.fm API Terms of Service, `https://www.last.fm/api/tos`. Accessed October 2026.